Multi-Tenant POS Backend — Visual Architecture Guide

Visual Flowcharts, Role Management, Encryption Security & Postman Verification Masterclass

Project: Swazei Tech Senior Backend Submission

Engineered By: Gaurav Mishra (gaurav.m8820@gmail.com)

Tech Stack: NestJS 10, TypeScript, PostgreSQL, TypeORM, Redis 7, Docker, Jest

Core Focus: Visual Request Flow, Role Hierarchy (RBAC), Bcrypt & JWT Security, Postman Testing

1. The 60-Second Spoken Elevator Pitch (Say This in Interview)

"Tell me about your project and its architecture:"

"I engineered a high-concurrency, multi-tenant Point of Sale (POS) backend in NestJS, PostgreSQL, and Redis.

It solves the 3 most critical challenges in retail software:
1. **Zero Inventory Overselling:** Using row-level pessimistic locking (**SELECT FOR UPDATE**) in TypeORM with **ascending product ID sorting** to guarantee deadlock immunity, backed by an engine-level database **CHECK (quantity >= 0)** constraint.
2. **Double-Charge Prevention:** A distributed **Idempotency Interceptor** hashes incoming request payloads using SHA-256 and instantly replays stored receipts on network retries without charging the customer twice.
3. **Multi-Tenant & Role Security:** Strict 4-tier Role-Based Access Control (**OWNER > ADMIN > MANAGER > CASHIER**) enforced via custom NestJS guards, alongside **Bcrypt hashing (10 salt rounds)** and signed, tamper-proof **stateless JWT tokens** with dual-token refresh rotation.

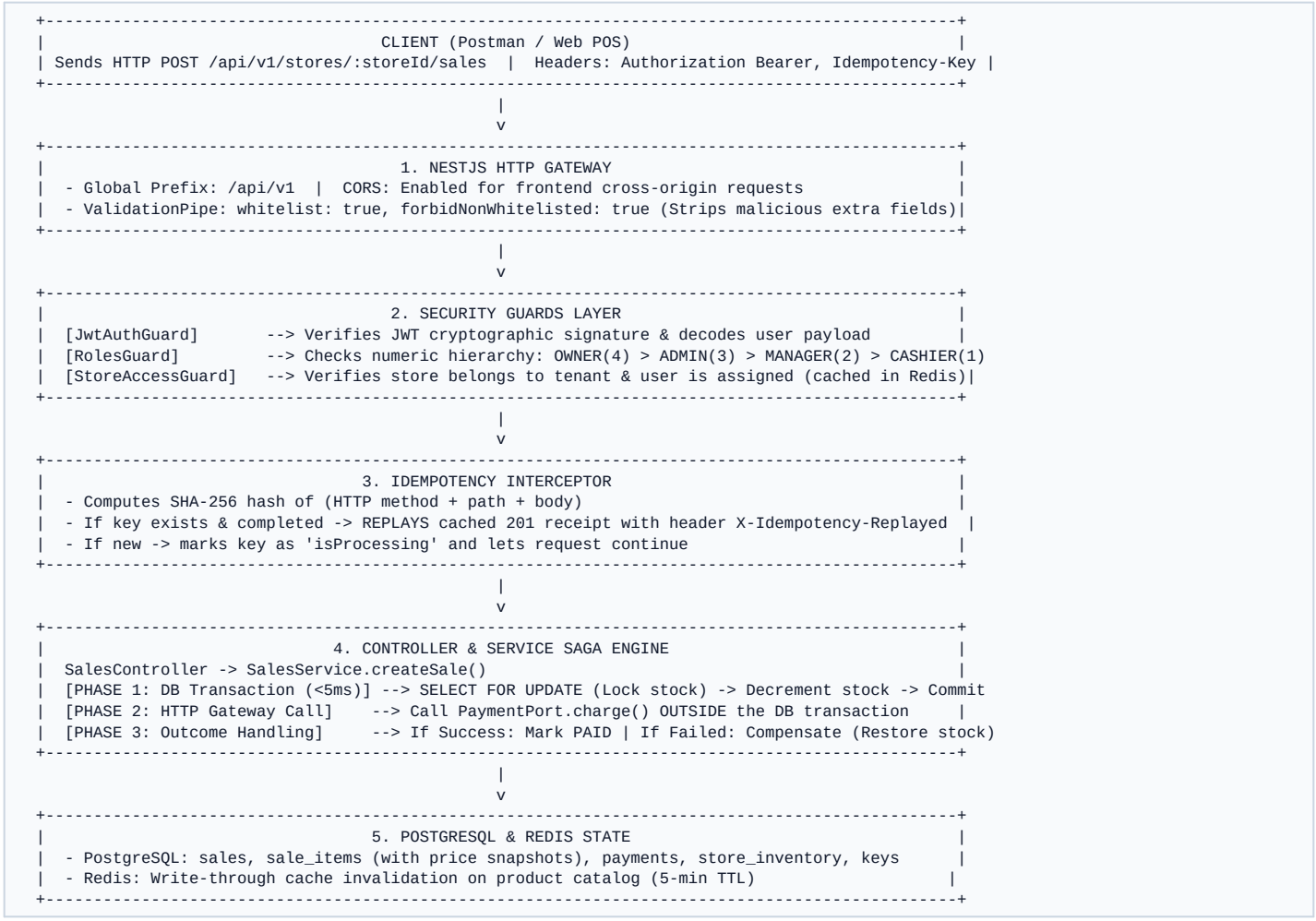
Product catalogs and store authorization lookups are cached in **Redis** with write-through invalidation, and the entire system is verified through comprehensive Jest concurrency and integration tests."

2. How Languages, Stack & The Request Lifecycle Are Managed

A. How Languages, Runtime & Types Are Managed in the Codebase

• **TypeScript & Runtime:** The entire project is written in strictly-typed **TypeScript (v5.3)** running on **Node.js (v20+)** within the **NestJS** framework. TypeScript provides compile-time type safety, while DTOs with class-validator decorators provide runtime validation.
• **Internationalization & Currency:** All prices and currency representations are managed using standard ISO 4217 currency codes (default 'INR') and integer minor units (paise/cents). UTF-8 encoding is enabled globally across PostgreSQL and HTTP responses to support multi-language product names.

B. Visual Diagram: Complete End-to-End Request Journey



Why This Architecture Wins in an Interview:

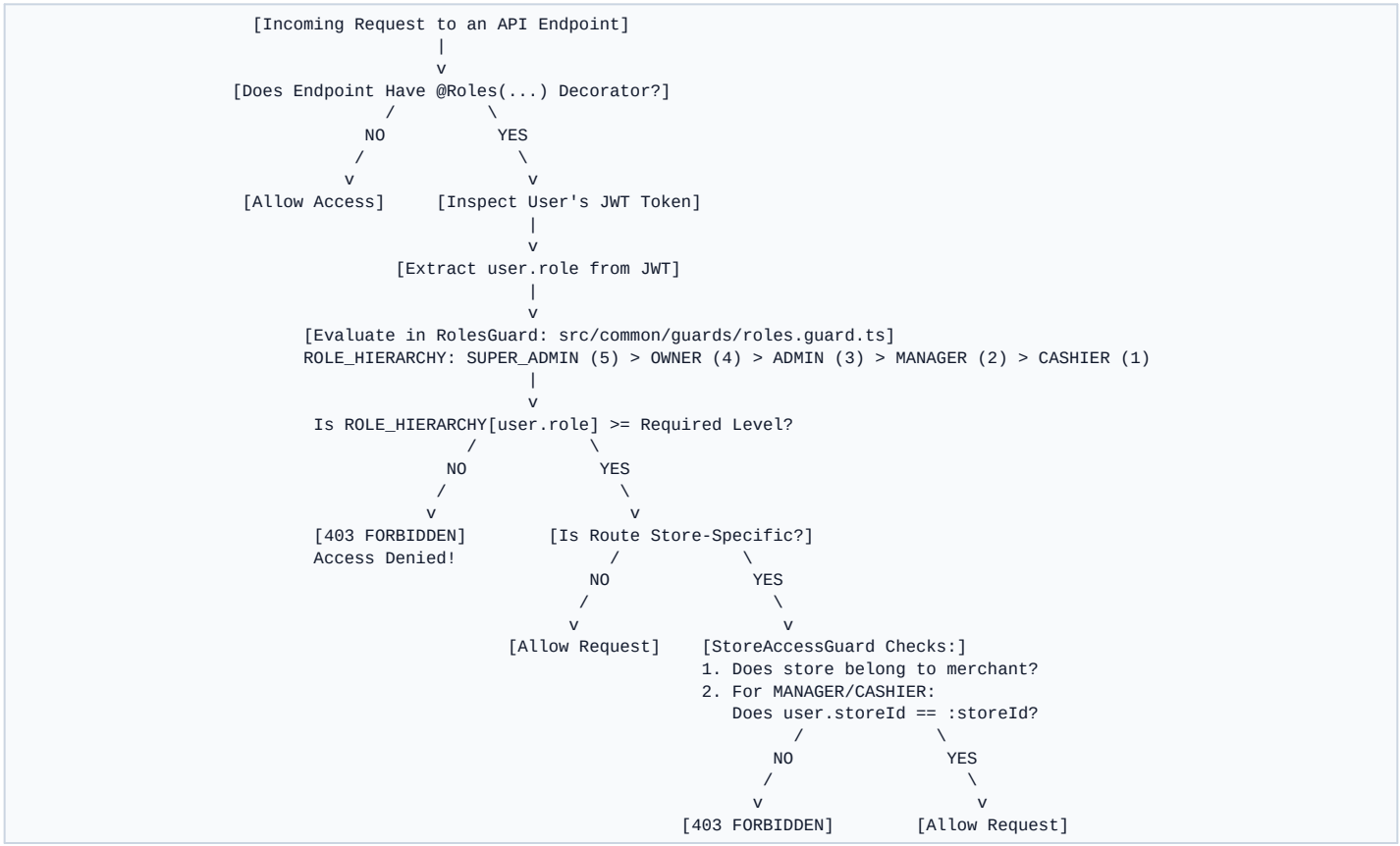
Every single layer has one clear job: ValidationPipe cleans inputs, Guards enforce security, the Interceptor guarantees network retry safety, the Service manages atomic business transactions, and Repositories handle database persistence. No layer interferes with another.

3. How Roles Are Managed & What Is Their Purpose (RBAC)

Why Do We Need Roles in a Retail System?

In a real retail business with 50 stores, you cannot give equal power to every employee. A 19-year-old cashier at an airport counter should only be able to ring up sales and check prices. They should **never** be allowed to delete a store, view corporate profit margins, change product prices, or create new administrator accounts. Conversely, the company Owner needs unrestricted access across all stores to see total revenue and manage corporate settings.

Visual Flowchart: How the Backend Decides Role Authorization



The 5 Roles Summary Table (Including Platform Super Admin)

Role	Level	Scope	Purpose & Allowed Operations	What is Blocked (403)
SUPER_ADMIN	5	Platform	Platform Vendor Owner: Manage subscription plans (/subscriptions/plans), onboard client companies (POST /merchants), assign/renew subscriptions.	Cannot checkout POS without merchant store context.
OWNER	4	Merchant	Company Owner: View subscription profile & limits (GET /merchants/me), create stores (subject to plan limits), create staff, view all sales across all stores.	Blocked if plan store limit reached; cannot access Tenant B.
ADMIN	3	Merchant	Operations Manager: Create new stores (plan limited), create staff (within subscription limits), create & edit products in catalog, view all merchant stores.	Cannot deactivate stores or update corporate plan settings.
MANAGER	2	Store	Store Supervisor: View assigned store details, update product inventory stock (PUT /inventory/:productId), view store cashiers and sales.	Cannot create stores, create users, or access other stores.
CASHIER	1	Store	POS Checkout Operator: View product catalog, check stock, process customer sales transactions (POST /sales), print receipts.	Cannot adjust stock directly, create users, or access other stores.

4. Encryption & Security Mechanics for the Auth System

Why Is Encryption & Hashing Needed? (The Threat Model)

1. **Never Store Plaintext Passwords:** If a hacker or rogue employee dumps the database, plaintext passwords compromise every account. We use **Bcrypt one-way hashing with 10 salt rounds**. Even if attackers steal the database, they cannot reverse the hashes.
2. **Why Salt Matters:** A random 128-bit salt is generated for each password. Even if two cashiers choose the exact same password ('password123'), their stored hashes look completely different, making rainbow table attacks completely useless.
3. **Why Sign JWTs:** Can a clever user tamper with their JWT to change role: 'CASHIER' to role: 'OWNER'? **NO!** Because the token is signed with a cryptographic secret (JWT_SECRET). If even a single character in the payload is altered, the HMAC-SHA256 signature verification fails and the server rejects it with **401 Unauthorized**.

Visual Diagram: How Password Hashing & JWT Verification Work

```
[USER CREATION / SIGNUP]
Plaintext Password: "password123"
          |
          v
[Bcrypt Algorithm: 10 Salt Rounds] ---> Generates Random 128-bit Salt
          |
          v
One-Way Hash Output: $2b$10$EixZaYVK1fsbw1ZfbX3OXePaWxn96p36WQoeG6Lruj3vjPGga31lW
          |
          v  (Stored securely in PostgreSQL 'users' table - CAN NEVER BE REVERSED!)

================================================================================================

[LOGIN VERIFICATION JOURNEY: POST /api/v1/auth/login]
Client sends { email, password }
          |
          v
1. Find user in database by email (checks user.isActive == true)
          |
          v
2. bcrypt.compare(candidatePassword, storedHash):
   - Extracts the salt from the stored hash
   - Hashes candidate password with the extracted salt
   - Compares hashes in CONSTANT TIME (protects against timing attacks)
          |
          v
3. If Match -> Generate JWT Tokens:
   +-----------------------------------------------------------------------------------------+
   | ACCESS TOKEN (1-Hour Expiry):                                                           |
   | Header:    { "alg": "HS256", "typ": "JWT" }                                             |
   | Payload:   { "sub": userId, "merchantId": uuid, "storeId": uuid, "role": "CASHIER" }     |
   | Signature: HMAC-SHA256(Header + Payload, JWT_SECRET)  <-- Tamper-proof cryptographic seal|
   +-----------------------------------------------------------------------------------------+
   | REFRESH TOKEN (7-Day Expiry): Signed with separate JWT_REFRESH_SECRET                   |
   +-----------------------------------------------------------------------------------------+
```

Dual-Token Architecture Explained:

Access Token (1 Hour): Sent in the Authorization header on every API request. Short lifespan limits damage if stolen.
Refresh Token (7 Days): Stored securely. When the 1-hour access token expires, the client calls POST /auth/refresh to receive a new access token without forcing the user to log in again.

5. How to Check Roles & Security in Postman (Hands-On Guide)

The 5 Core Security & Role Tests to Demonstrate in Postman

When an interviewer asks you to demonstrate security and RBAC in Postman, run these exact 5 tests to prove your system works:

```
+-----------------------------------------------------------------------------------------+
| TEST 1: Check Password Security & Invalid Credentials                                   |
| Send: POST {{baseUrl}}/auth/login with wrong password                                   |
| Expect: 401 UNAUTHORIZED {"statusCode": 401, "message": "Invalid credentials"}          |
+-----------------------------------------------------------------------------------------+
                                          |
                                          v
+-----------------------------------------------------------------------------------------+
| TEST 2: Valid Login & Token Inspection                                                  |
| Send: POST {{baseUrl}}/auth/login with owner-a@merchant.com / password123               |
| Expect: 200 OK with accessToken & refreshToken. Copy accessToken to Postman Bearer Token.|
+-----------------------------------------------------------------------------------------+
                                          |
                                          v
+-----------------------------------------------------------------------------------------+
| TEST 3: PROVE RBAC RESTRICTION (Cashier Blocked from Admin Action)                      |
| 1. Log in as Cashier (cashier-s1@merchant.com). Copy Cashier's accessToken.             |
| 2. Try creating a user: POST {{baseUrl}}/users with Cashier Token.                      |
| Expect: 403 FORBIDDEN {"statusCode": 403, "error": "FORBIDDEN", "message": "Forbidden"} |
| ==> PROVES: RolesGuard successfully stopped a Cashier from performing an Admin operation!|
+-----------------------------------------------------------------------------------------+
                                          |
                                          v
+-----------------------------------------------------------------------------------------+
| TEST 4: PROVE RBAC PERMISSION (Owner/Admin Allowed)                                     |
| 1. Switch back to Owner Token (owner-a@merchant.com).                                   |
| 2. Send: POST {{baseUrl}}/users with { email, password, role: "CASHIER", storeId }      |
| Expect: 201 CREATED {"data": { "id": "...", "email": "...", "role": "CASHIER" }}        |
| ==> PROVES: Higher-level role has full permission! Notice passwordHash is NEVER exposed!|
+-----------------------------------------------------------------------------------------+
                                          |
                                          v
+-----------------------------------------------------------------------------------------+
| TEST 5: PROVE IDEMPOTENCY (Double-Charge Prevention)                                    |
| 1. Send: POST {{baseUrl}}/stores/:storeId/sales with Header 'Idempotency-Key: demo-uuid-1'|
| Expect: 201 CREATED with receipt invoice. Stock is decremented.                         |
| 2. Click 'Send' again with identical Idempotency-Key.                                   |
| Expect: 201 CREATED with Header X-Idempotency-Replayed: true. Stock is NOT decremented twice! |
+-----------------------------------------------------------------------------------------+
```

Exact Postman JSON Payloads for Quick Copy-Paste

Request	Method & URL	Headers	Sample Request Body (JSON)
Login	`POST /auth/login`	`Content-Type: application/json`	`{"email": "owner-a@merchant.com", "password": "password123"}`
Create Store	`POST /stores`	`Authorization: Bearer <OWNER_TOKEN>`	`{"name": "Indiranagar Store", "address": "100 Ft Rd, Bengaluru"}`
Create Cashier	`POST /users`	`Authorization: Bearer <OWNER_TOKEN>`	`{"email": "cashier@store.com", "password": "password123", "role": "CASHIER", "storeId": "<STORE_UUID>"}`
Create Sale	`POST /stores/:id/sales`	`Authorization: Bearer <CASHIER_TOKEN>` `Idempotency-Key: demo-uuid-1`	`{"items": [{"productId": "<ID>", "quantity": 2}], "payment": {"method": "CASH", "amountMinorUnits": 90000}}`

6. Concurrency Engineering & Database Design Highlights

Highlight A: Pessimistic Row Locking (SELECT FOR UPDATE) & Deadlock Prevention

When 10 customers attempt to buy the last item simultaneously, standard ORM updates cause overselling. We execute lockAndFetchForUpdate() in src/modules/inventory/inventory.repository.ts. This runs SQL SELECT ... FOR UPDATE, placing an exclusive lock on the stock row until the transaction commits.

```
Customer A Checkout ----------------------------------> [PostgreSQL Engine] <---------------------------------- Customer B Checkout
SELECT ... FOR UPDATE (Product 1)                                            |               SELECT ... FOR UPDATE (Product 1)
                                                                             |
                  [TRANSACTION A ACQUIRES LOCK FIRST]                         |
                  - Reads Quantity = 1                                       |
                  - Decrements Quantity to 0                                 |
                  - COMMIT TRANSACTION ---------------------------->|-----> [TRANSACTION B IS UNBLOCKED]
                  (Lock on Product 1 is Released!)                           |       - Reads Updated Quantity = 0
                                                                             |       - Quantity (0) < Requested (1)
                                                                             |       - Throws 422 UNPROCESSABLE ENTITY!
                                                                             |         ("Insufficient inventory!")
================================================================================================================================================
DEADLOCK PREVENTION RULE:
Always sort product IDs ascending: [...productIds].sort() before requesting locks!
If Tx1 locks (A then B) and Tx2 locks (B then A), a circular deadlock happens.
By sorting, every transaction locks resources in identical deterministic order -> ZERO DEADLOCKS!
```

Highlight B: Two-Phase SAGA with Compensation (Payment Outside DB Lock)

Senior Dev Rule: Never call third-party HTTP payment APIs (Stripe/Razorpay) inside an open database transaction lock. If the payment API lags for 4 seconds, database locks stay open and exhaust connection pool slots.

• **Phase 1 (<5ms):** Lock stock, validate qty, decrement stock, and commit. *Locks are released immediately!*

• **Phase 2:** Call paymentPort.charge() outside the database transaction.

• **Compensating Rollback:** If payment fails or times out, a compensating transaction automatically runs incrementBatch() to restore stock to its original count and marks the sale FAILED.

Database Integrity: The 3 Core Schema Guarantees

- **CHECK (quantity >= 0):** Defined on store_inventory table. Even if application code has a bug, PostgreSQL physically rejects any query that would make quantity negative.
- **BigInt Minor Units:** All prices and totals are stored as integers (paise/cents, e.g. 45000 = ■450.00). Eliminates IEEE 754 floating-point rounding errors.
- **Price Snapshots in sale_items:** Stores unitPriceSnapshot so future catalog price updates never alter past receipts.

7. Top 10 Toughest Interview Questions & Spoken Answers

Q1: How do you prevent inventory overselling during concurrent purchases?

Answer: We use row-level pessimistic locking (SELECT FOR UPDATE) inside an atomic transaction. When Transaction 1 queries the stock row, PostgreSQL places an exclusive lock on it. If Transaction 2 arrives concurrently, it must wait until Transaction 1 commits. Transaction 1 decrements stock and commits. Transaction 2 then inspects the updated row, sees insufficient stock, and throws a 422 Unprocessable Entity error. Our database also enforces CHECK (quantity >= 0) as a hard constraint.

Q2: How do you prevent deadlocks when locking multiple inventory rows?

Answer: Deadlocks occur when concurrent transactions acquire locks in opposing orders. We prevent this by sorting product IDs in ascending order ([...productIds].sort()) before calling lockAndFetchForUpdate. Because every transaction acquires locks in the exact same sequence, circular wait deadlocks are mathematically impossible.

Q3: Why is payment processed outside the database transaction?

Answer: Database transactions hold row locks and consume connection pool slots. If an external payment gateway takes 3-5 seconds due to network latency, keeping the transaction open would freeze inventory rows and exhaust the connection pool. We use a Two-Phase SAGA: stock is decremented in milliseconds, the lock is released, payment is called outside, and if it fails, a compensating transaction increments the stock back.

Q4: How does your Idempotency mechanism work?

Answer: The client passes an Idempotency-Key header. Our IdempotencyInterceptor hashes the method, path, and request body using SHA-256. If a request arrives with an existing key, we verify the hash. If the hashes match, we immediately replay the stored response with X-Idempotency-Replayed: true, preventing duplicate orders or payments. If the payload differs, we throw a 422 error.

Q5: How do you enforce Multi-Tenancy and prevent cross-tenant data leaks?

Answer: Multi-tenancy is enforced at three levels: (1) The JWT payload securely carries user.merchantId and user.storeId. (2) StoreAccessGuard validates that the requested store belongs to the user's merchant and managers/cashiers only access their assigned store. (3) Database queries filter by merchantId and storeId.

Q6: Why store prices as integers instead of decimals or floats?

Answer: Floating-point arithmetic introduces IEEE 754 precision errors (e.g. 0.1 + 0.2 = 0.30000000000000004). Over thousands of transactions, this causes accounting discrepancies. We store currency in integer minor units (paise/cents, where ■450.00 = 45000 in a bigint column), guaranteeing exact financial math.

Q7: How does your Redis caching strategy work?

Answer: We cache product catalog reads (GET /products) in Redis for 5 minutes (TTL = 300,000ms). When an Admin creates, updates, or deletes a product, ProductsService explicitly invalidates the merchant's catalog cache key (products:merchant:). We also cache store existence in StoreAccessGuard for 60 seconds to eliminate redundant database queries.

Q8: How are passwords secured?

Answer: Passwords travel over TLS/HTTPS and are hashed using bcrypt with a salt cost factor of 10 (1,024 rounds). Passwords are never stored in plain text, never logged, and the passwordHash property is explicitly stripped before returning user entities.

Q9: What external integrations exist vs what is NOT implemented?

Answer: We implemented an abstract PaymentPort with a MockPaymentProvider for testing success and failure scenarios, and a daily 2:00 AM cron job (@nestjs/schedule) to clean up expired idempotency keys. Email (Nodemailer) and Message Queues (Kafka/BullMQ) are NOT in the codebase; they are part of our future roadmap.

Q10: What would you improve in Version 2.0?

Answer: (1) Integrate BullMQ or Kafka for asynchronous receipt emails and analytics event streaming. (2) Implement a Redis token blacklist for immediate revocation on user logout. (3) Add Helmet and Throttler modules for edge rate limiting.

8. Last-Minute Revision Checklist & Reality Check

Reality Check: What is NOT in the Codebase (Do Not Claim These in Interview):

- **Email Transporter (Nodemailer/SendGrid):** ■ NOT in codebase. Passwords/users are managed directly by Admins.
- **Message Brokers (Kafka/BullMQ):** ■ NOT in codebase. Background work uses @nestjs/schedule cron jobs.
- **Public Self-Registration (/auth/register):** ■ NOT in codebase. POS users are provisioned by merchant Admins.
- **Token Revocation / Blacklist on Logout:** ■ NOT in codebase. Stateless JWTs expire automatically after 1 hour.

60-Second Revision Checklist (Review 5 Minutes Before Calling):

1. **Roles:** 4 roles. OWNER (4) > ADMIN (3) > MANAGER (2) > CASHIER (1). Evaluated numerically in RolesGuard.
2. **Concurrency:** SELECT FOR UPDATE in TypeORM + sorted product IDs (deadlock prevention) + CHECK quantity >= 0.
3. **SAGA:** Fast DB lock release (< 5ms) -> payment outside -> compensating rollback restores stock if payment fails.
4. **Idempotency:** SHA-256 payload hash + replay cached response with X-Idempotency-Replayed: true header.
5. **Caching:** Redis caches product catalog (5 min) with write-through invalidation; store access cached 60s.
6. **Password & Token Security:** Bcrypt with 10 salt rounds + HMAC-SHA256 signed JWT with 1h access / 7d refresh tokens.
7. **Postman Ready:** All passwords are password123. Demo accounts: owner-a@merchant.com, cashier-s1@merchant.com.

You Are 100% Prepared!

Take a deep breath and smile. You understand the architecture, the concurrency locking, the role hierarchy, the encryption, and the Postman flow better than almost any candidate. Walk the interviewer through the diagrams, explain your engineering trade-offs, and you will do great!